영지식증명 톺아보기 7편: PLONK - 범용 SNARK의 시대

Author: 김대익
Reviewed By: —

Disclaimer: 본 글은 PLONK 증명 시스템을 다룹니다. 범용 Trusted Setup, PLONKish 산술화, Copy Constraint, Permutation Argument의 원리를 설명합니다.

---

1. 들어가며: 왜 PLONK인가

2019년 8월, Ariel Gabizon, Zachary Williamson, Oana Ciobotaru가 **PLONK**를 발표했습니다[^1].

> **P**ermutations over **L**agrange-bases for **O**ecumenical **N**oninteractive arguments of **K**nowledge

(다소 억지스러운 약어지만, 발음하기 좋습니다!)

[^1]: Gabizon, A., Williamson, Z., Ciobotaru, O. (2019). *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. ePrint 2019/953. https://eprint.iacr.org/2019/953

1.1 Groth16의 한계

5편에서 다룬 Groth16은 훌륭하지만:

회로별 Trusted Setup:

- 새 회로마다 새 세레모니 필요
- 참여자 모집, 검증, 시간 소요
- 회로 변경 = 처음부터 다시

실용적 문제:

- zkRollup이 로직 업그레이드하면?
- 새로운 스마트 컨트랙트마다 세레모니?
- 개발 주기와 맞지 않음

1.2 PLONK의 해결책

PLONK는 **범용(Universal) Trusted Setup**을 제공합니다:

1. **한 번의 Setup으로 모든 회로**
2. **업데이트 가능 (Updatable)**
3. **다항식 커밋먼트만 교체 가능**

이것이 "Oecumenical"의 의미입니다: 다양한 사용 사례를 포용합니다.

---

2. PLONK vs Groth16: 핵심 차이

2.1 Setup 비교

특성	Groth16	PLONK
Setup 유형	회로별 (Circuit-specific)	범용 (Universal)
새 회로	새 세레모니 필요	기존 SRS 재사용
업데이트	불가능	가능
SRS 크기	O(회로 크기)	O(회로 크기)

2.2 증명 크기 및 성능

메트릭	Groth16	PLONK
증명 크기	~200 bytes ($2G_1 + 1G_2$)	~400-900 bytes ($9G_1 + 7F$)
검증 시간	3 pairings	2 pairings + 추가 연산
온체인 비용	~270k gas	~300k gas
증명 생성	빠름	조금 느림 (일반적으로)

출처: SP1 Docs[^2], Aztec Benchmarks[^3]

[^2]: Succinct Docs. *Proof Types*. https://docs.succinct.xyz/docs/sp1/generating-proofs/proof-types [^3]: Aztec Labs. *PLONK Benchmarks*. https://medium.com/aztec-protocol/plonk-benchmarks-2-5x-faster-than-groth16-on-mimc-9e1009f96dfe

2.3 언제 무엇을 선택?

Groth16:

- 회로가 고정되고 자주 사용됨
- 증명 크기가 매우 중요
- 온체인 비용 최소화가 목표

PLONK:

- 회로가 자주 변경됨
- 여러 다른 회로를 지원해야 함
- 개발 유연성이 중요

---

3. PLONKish 산술화

3.1 R1CS vs PLONKish

Groth16은 **R1CS**를 사용합니다:

```
(A·s) ⊙ (B·s) = (C·s)
```

PLONK는 **게이트 기반** 시스템을 사용합니다:

```
q_L·a + q_R·b + q_O·c + q_M·(a·b) + q_C = 0
```

3.2 PLONKish 게이트 구조

각 게이트 (행 i)는 다음 제약을 만족해야 합니다[^4]:

```
q_L[i]·a[i] + q_R[i]·b[i] + q_O[i]·c[i] + q_M[i]·a[i]·b[i] + q_C[i] = 0
```

여기서:

- **a, b**: 좌/우 입력 와이어
- **c**: 출력 와이어
- **q_L, q_R, q_O, q_M, q_C**: 셀렉터 다항식

[^4]: Halo2 Book. *PLONKish Arithmetization*. https://zcash.github.io/halo2/concepts/arithmetization.html

3.3 게이트 타입 예시

덧셈 게이트: a + b = c

```
q_L=1, q_R=1, q_O=-1, q_M=0, q_C=0
→ a + b - c = 0 ✓
```

곱셈 게이트: a × b = c

```
q_L=0, q_R=0, q_O=-1, q_M=1, q_C=0
→ a·b - c = 0 ✓
```

상수 게이트: a = 5

```
q_L=1, q_R=0, q_O=0, q_M=0, q_C=-5
→ a - 5 = 0 ✓
```

3.4 테이블 형태

n개의 게이트가 있을 때:

행	a	b	c	q_L	q_R	q_O	q_M	q_C
0	a_0	b_0	c_0	...	...	...	...	...
1	a_1	b_1	c_1	...	...	...	...	...

행	a	b	c	q_L	q_R	q_O	q_M	q_C
...	...	...	...	...	...	...	...	...
n-1	...	...	...	...	...	...	...	...

와이어 다항식: a(X), b(X), c(X) - Witness (비공개) **셀렉터 다항식**: q_L(X), q_R(X), ... - Preprocessed (공개)

---

4. Copy Constraints와 Permutation Argument

4.1 Copy Constraints란?

게이트 제약만으로는 부족합니다. **와이어 연결**이 필요합니다[^5].

예시:

```
게이트 0: a₀ × b₀ = c₀
게이트 1: c₀ + b₁ = c₁   ← c₀을 입력으로 사용!
```

여기서 **게이트 0의 출력 c_0**이 **게이트 1의 입력 a_1**이 되어야 합니다:

```
c₀ = a₁  (Copy Constraint)
```

[^5]: sCrypt. *How PLONK Works: Part 2. Copy Constraints*. https://scryptplatform.medium.com/how-plonk-works-part-2-1072dcd7634a

4.2 Permutation으로 표현

3n개의 셀 (n 게이트 × 3 와이어)을 하나의 벡터로 펼칩니다:

```
[a₀, a₁, ..., a_{n-1}, b₀, b₁, ..., b_{n-1}, c₀, c₁, ..., c_{n-1}]
```

Copy Constraint는 이 벡터의 **순열(Permutation)**로 표현됩니다.

예: $c_0 = a_1$이면

- c_0의 인덱스: 2n + 0 = 2n
- a_1의 인덱스: 1
- σ(1) = 2n, σ(2n) = 1

4.3 Permutation Argument

목표: 벡터 v가 순열 σ 하에서 자기 자신과 같음을 증명

핵심 아이디어 (Bayer-Groth 2012):

두 다항식이 같으면, 랜덤 점에서의 평가도 같다 (Schwartz-Zippel).

```
∏ᵢ (v[i] + β·i + γ) = ∏ᵢ (v[i] + β·σ(i) + γ)
```

여기서:

- β, γ: 랜덤 챌린지
- i: 원래 인덱스
- σ(i): 순열된 인덱스

왜 이것이 작동하는가?

만약 v[i] = v[j]이고 σ(i) = j, σ(j) = i라면:

- 좌변의 i항: v[i] + β·i + γ
- 좌변의 j항: v[j] + β·j + γ
- 우변의 i항: v[i] + β·j + γ (σ(i) = j이므로)
- 우변의 j항: v[j] + β·i + γ (σ(j) = i이므로)

v[i] = v[j]이면 이 쌍이 교환되어도 곱은 같습니다!

4.4 Grand Product Argument

순열 체크를 **누적 곱**으로 변환합니다:

```
Z[0] = 1
Z[i+1] = Z[i] · (v[i] + β·i + γ) / (v[i] + β·σ(i) + γ)
```

검증:

- Z[0] = 1 (시작)
- Z[n] = 1 (끝 - 모든 항이 상쇄됨)
- 점화식이 각 i에서 성립

이 Z를 **다항식으로 인코딩**하고, 랜덤 점에서 검증합니다.

---

5. PLONK 프로토콜

5.1 전체 흐름

```
1. [전처리] 회로 → 셀렉터 다항식 커밋
2. [증명자] 와이어 다항식 커밋 (a, b, c)
3. [검증자 → 증명자] 챌린지 β, γ
4. [증명자] 순열 누적 다항식 Z 커밋
5. [검증자 → 증명자] 챌린지 α
6. [증명자] 모든 제약을 결합한 몫 다항식 t 커밋
7. [검증자 → 증명자] 평가 점 ζ
```

```
8. [증명자] 다항식들의 ζ에서의 값과 Opening Proof
9. [검증자] 페어링으로 최종 검증
```

5.2 핵심 다항식들

공개 (Preprocessed):

- q_L(X), q_R(X), q_O(X), q_M(X), q_C(X): 셀렉터
- S_σ1(X), S_σ2(X), S_σ3(X): 순열 인코딩

비밀 (Witness):

- a(X), b(X), c(X): 와이어 값
- Z(X): 순열 누적 곱

몫 (Quotient):

- t(X): 모든 제약이 0임을 증명

5.3 제약 결합

α로 모든 제약을 하나로 결합:

```
L(X) = (게이트 제약) + α·(순열 시작 제약) + α²·(순열 끝 제약) + α³·(순열 점화 제약)
```

L(X)는 모든 ω^i (i = 0, ..., n-1)에서 0이어야 합니다.

따라서:

```
L(X) = t(X) · Z_H(X)
```

여기서 Z_H(X) = X^n - 1 (소거 다항식, 단위근에서 0)

5.4 증명 구성

최종 증명 π:

- 다항식 커밋먼트들 (9개 G_1 점)
- 평가 값들 (7개 필드 원소)
- Opening Proofs

증명 크기: ~400-900 bytes (KZG 사용 시)

---

6. Kate (KZG) 커밋먼트와의 결합

6.1 왜 KZG인가?

PLONK는 **다항식 커밋먼트 방식에 독립적**입니다[^6].

- **KZG**: 작은 증명, Trusted Setup 필요
- **FRI**: STARK처럼, Trusted Setup 없음, 큰 증명
- **DARK**: Hidden-order groups 기반

대부분의 PLONK 구현은 **KZG**를 사용합니다.

[^6]: Vitalik Buterin. *Understanding PLONK*. https://vitalik.eth.limo/general/2019/09/22/plonk.html

6.2 KZG 복습

5편에서 다룬 것처럼:

커밋: $C = f(\tau) \cdot G$ **평가 증명**: $f(a) = v$임을 증명 **검증**: 페어링으로 확인

6.3 배치 Opening

PLONK는 여러 다항식을 **같은 점에서 열어야** 합니다.

ζ에서 $a(\zeta)$, $b(\zeta)$, $c(\zeta)$, ... 모두를 증명하려면?

선형 결합:

```
L(X) = a(X) + v·b(X) + v²·c(X) + ...
```

하나의 Opening Proof로 모든 평가 검증!

---

7. Custom Gates와 UltraPLONK

7.1 Custom Gates

기본 PLONK 게이트만으로는 효율이 낮을 수 있습니다.

예: 해시 함수의 XOR 연산

Custom Gate 아이디어:

- 자주 사용되는 연산에 특화된 게이트 추가
- 셀렉터 다항식 추가로 활성화

7.2 TurboPLONK / UltraPLONK

TurboPLONK (Aztec):

- MiMC 해시에서 Groth16보다 **2.5x 빠름**[^3]
- Custom Gates로 곱셈 체인 최적화

UltraPLONK:

- **Lookup Tables** 추가

- 범위 검사, 비트 연산 등에 효율적
- Plookup 프로토콜 통합

7.3 PLONKup

Plookup (2020)과 PLONK 결합:

테이블 T에서 값 v를 찾는 것을 증명:

```
v ∈ T
```

응용:

- 범위 검사: v ∈ [0, 2^k)
- 비트 분해
- 해시 S-box 룩업

---

8. Halo2와 PLONKish의 진화

8.1 Halo2

Zcash의 **Halo2**는 PLONKish의 확장입니다[^7]:

- PLONK 산술화 기반
- **Trusted Setup 제거** (IPA 사용)
- **재귀 증명** 지원
- Custom Gates + Lookups

[^7]: Zcash. *The halo2 Book*. https://zcash.github.io/halo2/

8.2 Halo2의 특징

열(Column) 유형:

- **Advice**: Witness 값 (비밀)
- **Instance**: Public 입력
- **Fixed**: 상수 / 셀렉터
- **Selector**: 게이트 활성화

영역(Region):

- 회로의 논리적 단위
- 셀 할당의 기본 단위

8.3 PLONKish 생태계

프로젝트	기반	특징
Aztec	PLONK/TurboPLONK	프라이버시 L2

프로젝트	기반	특징
zkSync	PLONK 변형	zkRollup
Polygon zkEVM	커스텀 PLONK	EVM 호환
Scroll	Halo2 기반	zkEVM
Taiko	PLONK 기반	zkRollup

---

9. PLONK 변형들

9.1 fflonk

fflonk는 PLONK의 검증 최적화 버전:

- Opening 수 감소
- 더 작은 증명
- 더 빠른 검증

9.2 HyperPlonk

HyperPlonk (2023):

- **다변수 다항식** 사용
- **선형 시간 Prover**
- 고차 Custom Gates 지원

9.3 Plonky2/Plonky3

Polygon의 **Plonky2**[^8]:

- PLONK + FRI (STARK 스타일)
- **Goldilocks 필드** (64비트)
- 매우 빠른 Prover
- 재귀 증명 지원

[^8]: Polygon Zero Team. *Plonky2*. https://docs.rs/crate/plonky2/latest/source/plonky2.pdf

---

10. 실전 비교: PLONK vs Groth16 vs STARK

10.1 종합 비교

메트릭	Groth16	PLONK	STARK
증명 크기	~200 B	~500 B	~50-200 KB
검증 시간	가장 빠름	빠름	중간
Prover 시간	중간	중간	빠름 (소필드)
Trusted Setup	회로별	범용	없음

메트릭	Groth16	PLONK	STARK
양자 저항	❌	❌	☑
유연성	낮음	높음	높음

10.2 실제 선택 기준

zkRollup 팀의 선택:

프로젝트	증명 시스템	이유
zkSync Era	PLONK 변형	유연성, 업그레이드
Polygon zkEVM	STARK + Groth16	빠른 prover + 작은 onchain
Scroll	Halo2	Trusted Setup 회피
StarkNet	STARK	양자 저항, 투명성

하이브리드 접근:

```
STARK proof → Groth16/PLONK wrap → 작은 onchain proof
```

---

11. 마치며: PLONK가 열어준 세계

PLONK는 **zkSNARK의 실용화**를 가속화했습니다:

달성한 것:

- **범용 Setup**: 한 번의 세레모니로 무한한 회로
- **모듈성**: 다항식 커밋먼트 교체 가능
- **확장성**: Custom Gates, Lookups, 재귀

생태계 영향:

- Aztec, zkSync, Polygon, Scroll 등 대부분의 zkRollup이 PLONK 계열 사용
- Halo2로 진화하여 Trusted Setup까지 제거
- "PLONK-ish"가 산술화의 새 표준

교훈:

- 완벽한 시스템은 없음: 트레이드오프 존재
- 범용성이 때로는 효율성보다 중요
- 모듈식 설계가 혁신을 가속화

다음 편에서는 **Halo2와 재귀 증명**을 다룹니다:

- IPA와 Trusted Setup 제거
- Accumulation Scheme

- 무한한 계산의 압축

시리즈 로드맵

1편: 비밀을 증명한다는 것 ✓
2편: 컴퓨터가 이해하는 수학 ✓
3편: 프로그램을 다항식으로 - 산술화 ✓
4편: Zcash Sprout - 최초의 프라이버시 암호화폐 ✓
5편: Groth16의 마법 ✓
6편: Filecoin의 PoRep - 저장 증명 ✓
7편: PLONK - 범용 SNARK의 시대 (현재) ✓
8편: Halo2와 재귀 증명
9편: STARK와 FRI
...

Appendix A: PLONK 게이트 제약 상세

A.1 기본 게이트 제약

각 행 i에서:

```
q_L(ωⁱ)·a(ωⁱ) + q_R(ωⁱ)·b(ωⁱ) + q_O(ωⁱ)·c(ωⁱ)
+ q_M(ωⁱ)·a(ωⁱ)·b(ωⁱ) + q_C(ωⁱ) = 0
```

여기서 ω는 n차 단위근 ($\omega^n = 1$)

A.2 순열 제약

시작:

```
L₁(X)·(Z(X) - 1) = 0
```

(L_1은 ω^0에서만 1인 Lagrange 기저)

점화:

```
Z(ωX)·(a(X) + β·σ_a(X) + γ)·(b(X) + β·σ_b(X) + γ)·(c(X) + β·σ_c(X) + γ)
= Z(X)·(a(X) + β·X + γ)·(b(X) + β·k₁·X + γ)·(c(X) + β·k₂·X + γ)
```

A.3 몫 다항식

```
t(X) = [게이트제약 + α·순열제약 + α²·공개입력제약] / Z_H(X)
```

$Z_H(X) = X^n - 1$

Appendix B: 주요 구현체

라이브러리	언어	특징
gnark	Go	Groth16 + PLONK, 고성능
halo2	Rust	Zcash, Trusted Setup 없음
plonky2	Rust	FRI 기반, 매우 빠름
barretenberg	C++	Aztec, TurboPLONK
snarkjs	JS	브라우저 호환

References

1. Gabizon, A., Williamson, Z., Ciobotaru, O. (2019). *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. ePrint 2019/953. https://eprint.iacr.org/2019/953
2. Succinct Docs. *Proof Types*. https://docs.succinct.xyz/docs/sp1/generating-proofs/proof-types
3. Aztec Labs. *PLONK Benchmarks*. https://medium.com/aztec-protocol/plonk-benchmarks-2-5x-faster-than-groth16-on-mimc-9e1009f96dfe
4. Halo2 Book. *PLONKish Arithmetization*. https://zcash.github.io/halo2/concepts/arithmetization.html
5. sCrypt. *How PLONK Works: Copy Constraints*. https://scryptplatform.medium.com/how-plonk-works-part-2-1072dcd7634a
6. Vitalik Buterin. *Understanding PLONK*. https://vitalik.eth.limo/general/2019/09/22/plonk.html
7. Zcash. *The halo2 Book*. https://zcash.github.io/halo2/
8. Polygon Zero Team. *Plonky2*. https://docs.rs/crate/plonky2/latest/source/plonky2.pdf
9. gnark Docs. *Proving Schemes and Curves*. https://docs.gnark.consensys.io/Concepts/schemes_curves
10. ZKP Labs. *Introduce PLONK*. https://zkplabs.network/blog/Introduce-PLONK-Revolutionizing-ZK-SNARK-Technology-for-Efficiency-and-Privacy
11. ZK-Garage. *PLONK Book*. https://github.com/ZK-Garage/plonk/blob/master/plonk-book/src/chapter_2.md
12. Alpen Labs. *Current State of SNARKs*. https://www.alpenlabs.io/blog/current-state-of-snarks